

Lite3 Pilot console — setup guide

Covers: **command buttons**, **two-joystick driving** and **camera setup**. Applies to the web console (port 8123) and its config.json.

1. Commands (buttons)

1.1 How the buttons are organised

Buttons are grouped exactly as configured:

- **Top row:** 🐕 STAND · 🪑 SIT · 🖐️ HELLO · 🕺 DANCE · 🔄 TURN OVER (built-in pose buttons)
- **Recovery (from upside down):** 🔄 Recover left · 🔄 Recover right
- **Poses & actions:** 🌀 Twist · 🌀 Twist jump · 🕺 Backflip · 🐕 Long jump (plus Hello/Dance/Turn over)
- **Modes:** 🎮 Manual mode · 📡 Move mode
- **Gaits:** 🐢 Slow · 🚶 Medium · ⚡ Fast · 🐛 Crawl

1.2 The posture rule (very important)

- STAND and SIT are **one and the same command** (0x21010202) — a **toggle**. Sending it while already standing makes the robot sit, and vice versa.
- Actions that need a specific pose declare it ("posture": "sit" or "stand"). The console only re-poses when it knows the state is wrong; with "force_posture": true it re-poses even when the state is unknown (used for rolls/jumps).
- The posture tracker resets on restart, so after starting the app: put the robot in the pose you want (or let the action's own pose step do it) and press the button.
- Velocity/driving commands are only honoured while the robot is **standing**; while sitting they are ignored silently.

1.3 Adding or editing a command (config.json)

Each button is one entry under "actions". Example:

```
"twist_jump": {
  "enabled": true,
  "note": "twist jump (0x2101020D - standing, moves dog)",
  "posture": "stand",
  "force_posture": true,
  "replay": "slow",
  "group": "action",
  "icon": "🌀",
  "label": "Twist jump",
  "frame": { "code": "0x2101020D", "type": 0, "value": 0 }
}
```

- frame.code — the 32-bit command (hex). type 0 = simple frame; value — integer parameter (velocity frames use payload instead).
- posture / force_posture — pose handling as above. Omit for actions valid in any pose.

- replay: "slow" — sends the frame three times at ~ 1 Hz (used for poses/rolls). "once": true — a single frame (used for mode/gait switches). Default — short burst.
- group — which block the button appears in (recovery / action / mode / gait). icon + label — button face; note — tooltip.
- After editing: restart the console (or reload the page) — buttons appear automatically. Disable a button with "enabled": false.

1.4 Finding the right code for a new command

1. **Probe:** Advanced $\rightarrow$ raw frames — send a frame once and watch the robot.
2. **Capture from the official remote:** run tools/retroid_listen.py on the console computer, point the RETROID controlapp at <console IP>:12121, press the button there, and read the exact frame.
3. **Safety:** test new poses on a soft surface with clear space; keep the physical E-stop in reach. E-STOP in the console cancels any action mid-play and blocks all commands until ↻ Return.

2. Two-joystick movement

2.1 Layout

- **Left stick** — up/down = forward/back · left/right = turn.
- **Right stick** — left/right = slide (side-walk) · up/down = forward/back (if both sticks push forward, the stronger one wins).
- Diagonals combine axes (e.g. forward + slide, or forward + turn).
- Both sticks are **spring-centred**: release = knob returns to centre and the robot stops (a brake frame is sent immediately).

2.2 Stopping

- Release either stick (or return it to centre) $\rightarrow$ stop.
- **STOP** button $\rightarrow$ official brake frames. **E-STOP** $\rightarrow$ latches until ↻ Return.
- Switching browser tab / losing window focus also stops the robot (safety).

2.3 Sensitivity and speed (Advanced)

- **Stick curve** — Linear / Soft (default) / Soft+: higher curve = finer control near the centre.
- **Speed scale** — 50 % / 75 % / 100 % / 150 %: multiplies the configured maxima (defaults: 0.5 m/s forward, 0.4 m/s lateral, 0.8 rad/s turn).
- **Forward / Turn / Lateral sign** — flips each axis if a direction feels reversed; applied instantly.
- **Joystick mode** — *Velocity frames* (proven) or *Retroid / official SDK* (sends the same 42-byte frames as the physical remote to UDP 12121).

2.4 Keyboard (alternative to the sticks)

- **W / S** (or $\uparrow/\downarrow$) — forward / back · **A / D** (or $\leftarrow/\rightarrow$) — turn · **Space** — E-stop (hold) · leaving the keys stops the robot.

3. Camera setup

3.1 How the video path works

- The console asks the robot to start its camera services (command 0x21012109, value 0x40, to UDP 43899) and reads the stream at `rtsp://<robot IP>:8554/test`, re-publishing it to the browser as MJPEG.
- Feed opens automatically after connecting and is kept open by a watchdog that retries every ~8 s (re-sending the start command, reconnecting the stream).

3.2 If the robot publishes nothing (RTSP 404 / no video)

1. Confirm the robot is on and reachable: ping 192.168.2.1 (robot Wi-Fi) or 192.168.1.120 (LAN).
2. Check the RTSP port is open and whether the stream name exists:
`nc -z -w2 192.168.2.1 8554`
`ffprobe -v error -rtsp_transport tcp -show_entries format=format_name -of csv=p=0 rtsp://192.168.2.1:8554/test`
A working stream prints `rtsp`; "404 Not Found" means the camera service on the robot is not publishing.
3. If the port is open but the stream is absent, the robot's camera service must be started on the robot (its own service/tooling); power-cycling the robot usually restores it.

3.3 Tuning quality for a stable feed (Advanced → Camera stream)

- **FPS** — 4–6 fps is plenty for driving (lower = lighter).
- **Scale** — e.g. 480:-1 (default, light) or 640:-1 (sharper, heavier); 320:-1 for the lowest bandwidth.
- **Quality** — MJPEG -q:v: 2 = best/largest, 18 = smallest; default 11 balances both.
- **Apply & restart camera** — applies immediately; **Camera off** stops the feed.
- Rule of thumb: if the picture stutters on the robot's Wi-Fi, lower FPS first, then scale, then raise the quality number.

Ports: commands 43893 (UDP) · telemetry 43897 (UDP) · camera control 43899 (UDP) · RTSP 8554 (TCP, /test) · remote 12121 (UDP) · console 8123 (HTTP).